Machine Learning Course 2026 Autumn: Experiment 2

October 13, 2026

1 Overview

In this experiment, you will implement linear and nonlinear Support Vector Machine (SVM) models on given datasets, and deepen your understanding of the hyperparameters and kernel functions through visualization. You will also implement a linear SVM using **NumPy** and compare it with the model in ***sklearn***. Tasks 1–4 are required; Task 5 is optional.

2 Dataset Description

The datasets are ready for you and are stored in the files ***T1_data.csv*** and ***T2_data.csv***. Each row is a sample, with the last column being the label and the other columns being the features. T1 has 51 samples and T2 has 200 samples; both have two features and labels -1 and $+1$. You also need to load the **Wisconsin Breast Cancer Dataset** using ***sklearn***: 569 samples, 30 features, and original labels 0 (malignant) and 1 (benign).

3 Task Description

3.1 Task 1

Given the ***T1*** dataset, fit a **linear SVM** with $C = 1$. Visualize the data, decision boundary, margin contours and support vectors. Compare $C \in \{0.01, 0.1, 1, 10, 100, 1000\}$, and explain how C changes the margin and the penalty for margin violations. These full-dataset plots illustrate the fitted models; training accuracy does not estimate unseen-data performance.

3.2 Task 2

- Given the ***T2*** dataset, fit a **nonlinear SVM with Gaussian (RBF) kernel**, initially with $C = 1$ and $\gamma = 1$. Visualize the nine combinations of $C \in \{0.1, 1, 10\}$ and $\gamma \in \{0.1, 1, 10\}$. Explain

their different effects on the decision boundary. The kernel is

$$K(x, z) = \exp\left(-\gamma \|x - z\|^2\right), \qquad \gamma = \frac{1}{2\sigma^2}.$$

Larger γ corresponds to a narrower Gaussian kernel. In the soft-margin dual, the constraints are $0 \leq \alpha_i \leq C$ and $\sum_i \alpha_i y_i = 0$.

- Tune C and γ using a **grid search**. Use all 21 values $2^{-10}, 2^{-9}, \ldots, 2^{10}$ for each parameter, giving 441 candidates. Use **5-fold stratified cross-validation**, with shuffle enabled, random state 42 and accuracy scoring. Report the optimal parameters and mean validation accuracy. Plot the 21-by-21 score heatmap with labelled logarithmic parameter axes. The best CV score is a tuning result, not an independent test score. Other kernel comparisons are optional.

3.3 Task 3

- Load the **Wisconsin Breast Cancer Dataset** as a feature DataFrame and target Series. Split it into a **training set (80%)** and a **test set (20%)** with stratification and random state 42. Keep raw features in `X_train` and `X_test`. Demonstrate standardization on training data in a separate variable and explain why feature scaling matters for SVMs.

- For model selection, use a **Pipeline** consisting of **StandardScaler** and an **RBF SVC**. The scaler must be fitted inside each training fold, rather than before cross-validation. Search exactly seven values $10^{-3}, 10^{-2}, \ldots, 10^{3}$ for C and seven values $10^{-4}, 10^{-3}, \ldots, 10^{2}$ for γ. Use shuffled, stratified five-fold CV, random state 42, and **malignant F1** scoring with `pos_label=0`. Fit the search only on the raw training subset.

- After selecting the parameters using CV, evaluate the best Pipeline once on the test set. Report **accuracy, precision, recall and F1**. Retain the original labels: **0 = malignant, 1 = benign**; malignant is the positive class for the binary metrics. Include both class reports and a labelled confusion matrix in order $[0, 1]$. Explain the malignant false negatives (true malignant predicted benign) and the difference between tuning CV and independent test performance.

3.4 Task 4

Use the provided data preparation, model interface and training-loop framework to implement a **linear SVM using NumPy**. The objective is

$$L(w, b) = \frac{1}{2}\|w\|^2 + C \sum_i \max\left(0, 1 - y_i(w^T x_i + b)\right).$$

The hinge penalties are **summed**, not averaged, and the bias is not regularized.

- Reload T1 and standardize it with NumPy means and population standard deviations, replacing any zero standard deviation by 1. Complete the loss, subgradients, parameter updates and prediction functions in `NumPyLinearSVM`. Provide `fit`, `decision_function` and `predict`; predict $+1$ when the decision score is zero. Derive both subgradients in your report.
- Initialize weights and bias to zero. Perform **5000 full-batch updates**. Use subgradients and the learning rate $\eta_t = 0.05/\sqrt{t+1}$ for $t = 0, \ldots, 4999$. Samples with $y_i(w^T x_i + b) < 1$ are active; choose the inactive branch at exactly margin 1. Save the training loss history and the parameters with the lowest objective, including the initial state. Run the supplied numerical gradient and score-zero checks.
- For $C \in \{0.1, 1, 10\}$, compare with `SVC` using `kernel="linear"`, the same C, and `tol=1e-8`. Use the same standardized data for both models. Report training accuracy, objective value and margin width $2/\|w\|$ in standardized coordinates. Losses must be finite, the best loss must improve on initialization, and the relative objective difference from sklearn must be at most 5%.
- Plot the raw and best-so-far loss curves and six decision boundaries. Analyze the differences caused by approximate optimization. Identical coefficients and monotonic decrease of the raw loss are not required.

3.5 Task 5 (Optional)

- Reload T2 and make a new stratified 80%/20% split with random state 42 (160 training and 40 test samples). Do not reuse the model or selected parameters from Task 2, whose search used all T2 samples.
- Use a StandardScaler–RBF SVC Pipeline with $C = 1$ and $\gamma \in \{0.01, 0.1, 1, 10, 100\}$. On the training subset only, use shuffled, stratified five-fold CV with random state 42. Report mean fold training accuracy, mean validation accuracy and validation standard deviation. Plot the training and validation curves against logarithmic γ, with validation variability. Also fit each candidate on the full training subset to report its number of support vectors.
- Select γ solely by the highest mean validation accuracy; ties use the first candidate in the listed order. Freeze the choice and evaluate only the selected model once on the independent test subset.
- Show the smallest, selected and largest γ boundaries using training data. Analyze underfitting, overfitting, the training/validation gap and support-vector counts without assuming monotonic results. Discuss the uncertainty of a 40-sample test set: one changed prediction changes its accuracy by 2.5 percentage points. Leave `RUN_TASK5=False` if you skip this task.

4 Requirements

- Implement learning algorithms with **Python**. Basic data analysis libraries such as **NumPy**, **Pandas**, **Matplotlib**, etc., are allowed to use. Tasks 1–3 and 5 may use ***sklearn***. In Task 4, the training algorithm, loss, gradients and predictions must use **NumPy only**; sklearn is allowed solely for the comparison model.
- Record the procedure of experiments in **Jupyter Notebook**. Use **Markdown** to explain your methods and results in English. Include the requested figures and tables with labelled axes and captions.
- Restart the kernel and run the completed notebook sequentially before submission. Record package versions, random seeds and CPU execution times. The provided settings use two search workers and 160-by-160 visualization meshes; no GPU is required.
- Your submission includes:
  - source code: the completed Python notebook (*.ipynb*) with executed outputs;
  - *report.pdf* file: your English report. Refer to *Template.doc* for the format, and clearly state whether you completed Task 5.
- The deadline is **November 11, 2026**.